DOG ADOPTION MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

MUKHESH L

(2116241501121)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "DOG ADOPTION MANAGEMENT SYSTEM" is the bonafide work of MUKHESH L (2116241501121), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr Ayyadurai M
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms Priya M
COURSE FACULTY
Assistant Professor
Department of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on Not evidenced.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, MUKHESH L (2116241501121), declare that this project report titled "DOG ADOPTION MANAGEMENT SYSTEM," submitted for the End Semester Examination of CB23F35 – Low-Code Application Development using Zoho Creator, records the individual project work completed by me. The application configuration and implementation described here are based on the supplied Zoho Creator application export and screenshots. Any external references used are cited in the report.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit.

Place: Chennai
Date: 26 October 2026

SIGNATURE OF THE STUDENT
Mukhesh L
2116241501121

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr Meganathan M, Chairperson, Ms Thangam Meganathan and Vice-Chairman, Mr Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr Murugesan SN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank, Dr Ayyadurai M, Head of the Department, Department of Artificial Intelligence and Machine Learning for the guidance and support provided throughout the course.

I sincerely thank my course faculty, Ms Priya M, Assistant Professor, for her guidance, reviews and feedback during the development of this application and report.

I also acknowledge Zoho Corporation / Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

Mukhesh L

ABSTRACT

The Dog Adoption Management System is a web-based application designed to simplify and manage the dog adoption process by connecting adopters, shelters/rescuers, and administrators through a single platform. It provides secure role-based login and separate dashboards for each user, ensuring that users can access only the information relevant to their role. The system allows adopters to browse available dogs, submit adoption applications, and track their application status, while shelters can manage dog listings and view applications for their own dogs.

The application includes essential modules such as adopter registration, shelter/rescuer registration, dog profile management, adoption applications, adoption records, follow-up feedback, and adoption events. Automated workflows improve efficiency by sending email notifications, preventing duplicate applications, updating dog availability after approvals or rejections, and scheduling follow-up reminders after successful adoptions. These features help reduce manual work and ensure a transparent adoption process.

The system also provides powerful dashboards and reports for administrators, including statistics on total dogs, adoptions, pending applications, and upcoming follow-ups, along with graphical reports such as bar, line, and pie charts. Built with a clean, mobile-friendly interface using a warm orange and white theme, the Dog Adoption Management System promotes responsible pet adoption by creating a secure, organized, and user-friendly experience for all stakeholders.

Keywords: Low-Code Development; Zoho Creator; Dog Adoption Management; Deluge; Workflow Automation

PROJECT ATA GLANCE

Attribute	Details
Project Title	Dog Adoption Management
Application Domain	Animal Welfare and Pet Adoption Management
Student Name & Register No.	Mukhesh L – 2116241501121
Zoho Creator Edition	Not identified in supplied export
Application Name (Link Name)	Dog Adoption Management System (Dog_Adoption_Management_System)
Published / Live URL	URL not recorded in report materials
Demo Credentials (per role)	Not included; use assigned Creator accounts
Number of Forms / Subforms	7 forms / no subforms evidenced
Number of Reports (by type)	4 register and grouped reports
Number of Pages / Dashboards	1 dashboard page evidenced
Number of Workflows	1 form workflow; 1 schedule; Blueprint stages
Deluge Functions / Custom Actions	Return notification and warranty reminder; no standalone function evidenced
Integrations	No external integration evidenced; built-in sendmail actions present
Roles & Profiles Defined	Adopter Profile,Shelter/Rescuer Profile,Dog Profile
Environments Used	Not documented
Date of Production Deployment	Not documented

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 4
2.3 Low-Code versus Traditional Development..... 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 7
REQUIREMENT ANALYSIS..... 8
3.1 Stakeholders and User Roles 8
3.2 Functional Requirements 8
3.3 Non-Functional Requirements 10
The project is implemented as a Zoho Creator application and accessed through a modern web browser. Deluge is the scripting language used by Creator. The screenshots show a web interface; no specific browser version, device specification, API testing tool, or mobile client test was recorded. 10
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 12
4.3 Form Design 13
4.4 Process and Workflow Design 14
4.5 User Interface and Navigation Design 15
4.6 Access Control Design 15
IMPLEMENTATION 16
5.1 Application Setup 16

5.2 Forms and Data Capture 16
5.3 Reports 17
5.4 Pages and Dashboards 19
5.5 Deluge Scripting 20
5.6 Workflows and Automation 22
5.7 Integrations 23
5.8 User Management and Role-Based Access 24
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 26
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 28
6.4 Defect Log 28
6.5 User Acceptance Testing 29
DEPLOYMENT, SECURITY AND GOVERNANCE..... 30
7.1 Environments and Deployment Strategy 30
7.2 Performance Optimisation 31
7.3 Security Features 32
7.4 Governance and Administration 32
RESULTS AND DISCUSSION..... 34
8.1 Application Output 34
8.2 Achievement of Objectives 35
8.3 Comparison with the Existing System 36
8.4 Limitations of the Low-Code Approach 36
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum..... 36
CONCLUSION AND FUTURE ENHANCEMENTS 38
9.1 Conclusion 38
9.2 Future Enhancements 38
9.3 Learning Outcomes and Career Relevance 38
REFERENCES 39
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 40
APPENDIX B – COURSE OUTCOME MAPPING 43
APPENDIX C – COMPLETE DELUGE SOURCE CODE 44
APPENDIX D – USER MANUAL 46
D.1 Accessing the Application 46

D.2 Adopter User Manual..... 46
D.3 Shelter/Rescuer User Manual 46
APPENDIX E – EVALUATION RUBRIC 47
APPENDIX F – VIVA-VOCE RECORD SHEET 49

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application……..3
Table 2.2: Summary of Literature Survey……..4
Table 3.1: Stakeholders and User Roles……5
Table 3.2: Functional Requirements……...5
Table 3.3: Non-Functional Requirements……...5
Table 3.4: Hardware and Software Requirements……..6
Table 3.5: Use Case Description – Not evidenced……6
Table 4.1: Entities and Relationships……9
Table 4.2: Field Specification – Not evidenced Form……9
Table 4.3: Role–Permission Matrix……..10
Table 5.1: Reports Implemented……..12
Table 5.2: Deluge Scripts Implemented……...13
Table 5.3: Workflows Implemented……14
Table 5.4: Integrations Implemented……15
Table 6.1: Test Cases……18
Table 6.2: Defect Log ……..19
Table 6.3: User Acceptance Feedback……19
Table 7.1: Deployment Checklist……20
Table 7.2: Performance Optimisation Results……..20
Table 7.3: Security Controls Implemented……...21
Table 8.1: Mapping of Objectives to Outcomes……...23
Table 8.2: Existing System versus Proposed Application……23
Table A.1: Feature Coverage Checklist……26
Table B.1: Mapping of Course Outcomes to Report Chapters……28
Table E.1: Evaluation Rubric for the Individual Project……..31
Table E.2: Performance Level Descriptors……...32
Table F.1: Viva-Voce Questions and Responses……..33
Table F.2: Marks Summary……..33

LIST OF FIGURES

Figure 2.1: Manual IT Asset and Request Process Used as the Baseline 4
Figure 3.1: User Roles and Main Application Use Cases 11
Figure 4.1: Zoho Creator Application Architecture 12
Figure 4.2: Core Entity Relationships in the Application 13
Figure 4.3: Asset Return Notification and Warranty Reminder Processes 15
Figure 4.4: Application Navigation and Dashboard 15
Figure 5.1: Application Dashboard and Navigation 16
Figure 5.2: IT Request Form 17
Figure 5.3: Asset Assignment and Asset Forms..... 17
Figure 5.4: Request and Assignment Reports 19
Figure 5.5: Dashboard and Asset Categories 19
Figure 5.6: Request Status, Employee and Workflow Configuration 23
Figure 5.7: No External Integration Configuration Supplied..... 24
Figure 5.8: Employee Role Form and Department Report 25
Figure 5.9: Dashboard Charts and Cost Summaries 26
Figure 5.10: Employee Directory; Mobile and Portal Access Not Verified 26
Figure 6.1: Request Status Board; No Debug Log Supplied 28
Figure 7.1: Live Dashboard; Environment History Not Supplied 30
Figure 7.2: Audit Trail Not Verified; Blueprint Shown in Figure 5.6 33
Figure 8.1: Maintenance Record Form 34
Figure 8.2: Workflow Configuration 35
Figure 8.3: Maintenance Type Board 35

LIST OF ABBREVIATIONS

This list defines abbreviations used in this report.

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
IT	Information Technology
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zoho Creator	Low-code application platform

CHAPTER 1
INTRODUCTION

1.1 Background

Introduce the domain and the organisation (real or realistic) for which the application is built. Explain how the process works today and who is involved. 1–2 pages.

The Dog Adoption Management System is a web-based application designed to simplify and manage the dog adoption process efficiently. It connects adopters, shelters/rescuers, and administrators through a single platform. The system allows adopters to browse available dogs, submit adoption applications, and track their application status.Shelters and rescuers can manage dog profiles, update health details, and view adoption requests, while administrators can verify users, approve dog listings, manage adoption records, and generate reports. The system also provides automated email notifications, follow-up reminders, and adoption event management. Overall, it reduces manual work, improves coordination, and promotes responsible dog adoption and animal welfare.

1.2 Problem Statement

State the problem precisely in one paragraph, followed by 3–5 specific pain points (e.g., data duplication across spreadsheets, delayed approvals, no real-time visibility, manual reporting, no access control).

The existing dog adoption process often relies on manual records, spreadsheets, phone calls, and emails to manage dog profiles, adopter details, and adoption applications. This makes it difficult to maintain accurate information, process applications quickly, and monitor adoption progress. Therefore, a centralized Dog Adoption Management System is needed to automate adoption activities, improve communication, protect user data, and ensure efficient coordination between adopters, shelters/rescuers, and administrators.

Key Pain Points

1. Data Duplication: Maintaining records across multiple spreadsheets and documents can lead to duplicate or inconsistent information.
2. Delayed Approvals: Manual verification of adopters, shelters, dog listings, and applications can slow down the adoption process.
3. Lack of Real-Time Visibility: Users may find it difficult to track application status, dog availability, and adoption progress.
4. Manual Reporting and Follow-Ups: Preparing adoption reports and remembering follow-up dates require extra time and effort.
5. Insufficient Access Control: Without role-based permissions, sensitive adopter information may be exposed to unauthorized users.

The objectives below describe the functions implemented or evidenced in the supplied application.

1. To develop a centralized platform connecting adopters, shelters/rescuers, and administrators.

2. To manage dog profiles, user registrations, and adoption applications efficiently.

3. To automate application notifications, approval processes, and post-adoption follow-up reminders.

4. To ensure secure role-based access and promote responsible dog adoption and animal welfare.

1.4 Scope of the Project

In scope are the five forms, their related lookups, eight reports, one dashboard page, an asset-return manager notification workflow, a warranty reminder schedule, request status stages and the Asset Maintenance Lifecycle blueprint. The screenshots show a web application interface. Mobile-specific behavior, customer portal access, production release, enterprise role permissions and third-party integrations are outside the verified evidence for this report.

Not evidenced

1.5 Justification for a Low-Code Approach

Zoho Creator suits this prototype because its visual form and report builders support the linked records and status views required by the project, while Deluge can extend event-driven behavior for email notifications. A blueprint provides a visual way to represent ordered process stages. This combination lets the project demonstrate data capture, reporting and workflow automation without building a separate web stack. The platform's managed hosting and mobile capabilities are platform features; the supplied evidence does not establish that mobile access or production governance was configured in this application [1], [2].

Not evidenced

1.6 Organisation of the Report

Chapter 2 studies the manual baseline, low-code development and related sources. Chapter 3 records project requirements. Chapter 4 presents the data model, processes and access design. Chapter 5 describes the Zoho Creator implementation. Chapter 6 defines the verification evidence and test plan. Chapter 7 documents deployment, security and governance status. Chapter 8 reviews the demonstrated outputs and limitations. Chapter 9 concludes the report and lists future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The baseline used for comparison is a typical manual process for a small internal IT team; no separate legacy application or measured baseline was supplied. An employee reports a fault or requests equipment by email or message. IT staff identify the requester and equipment, enter or update a spreadsheet, coordinate approval, arrange allocation or repair, and later update the record. Managers request status summaries, which staff prepare by filtering or combining files. This process description is a project assumption used for analysis, not an audited description of a named organization.

Adopter Contacts Shelter → Shelter Staff Collect Details → Staff Update Dog Information in Spreadsheet → Adoption Application Received → Application Reviewed → Approval Coordinated → Adoption Records Updated → Administrator Generates Reports

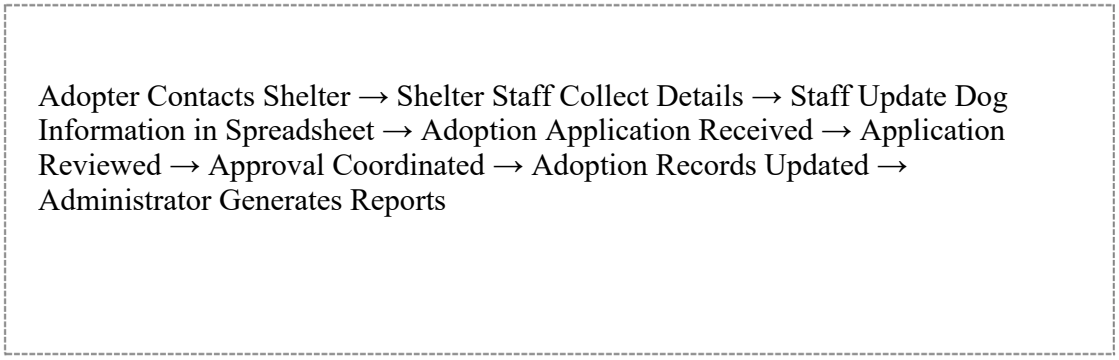


Figure 2.1: Manual Dog Adoption and Request Process Used as the Baseline

2.2 Limitations of the Existing System

- Manual maintenance of dog and adopter records is time-consuming and may cause errors.
- Information is scattered across spreadsheets, emails, and paper records.
- Adoption updates and post-adoption follow-up reminders may be delayed or missed.

2.3 Low-Code versus Traditional Development

For this project, Zoho Creator low-code development provides reusable form, lookup, report and workflow configuration in one application. A traditional

application would require separate implementation of its database, user interface, hosting, authentication and deployment pipeline. The low-code route reduces the amount of infrastructure code but still requires careful data modeling, Deluge scripting, permission review and platform-specific configuration. No comparative development-time measurement was recorded; therefore the comparison is qualitative.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Separate implementation of data, UI, hosting and workflows; no estimate measured.	Visual form, report and workflow configuration reduces custom infrastructure code; no estimate measured.
Skills required	Programming, database, web and deployment skills.	Creator configuration, data modeling and Deluge skills.
Hosting & infrastructure	Selected and maintained by development team.	Provided as part of Creator; app-specific setup still required.
Mobile application	Requires responsive UI or separate client work.	Creator offers mobile access; this project tested web screens only.
Security & access control	Implemented through selected identity and authorization components.	Platform supports permissions; project permission profiles not verified.
Maintenance & change	Changes require code, build and deployment practices.	Forms and reports can be changed visually; Deluge and platform dependencies remain.
Customisation limits	Broad control over architecture and UI.	Faster configuration within platform capabilities and limits.
Cost	Depends on engineering, hosting and maintenance choices.	Depends on subscription and configuration effort; actual project cost not recorded.

2.4 Literature Survey

The survey below uses two peer-reviewed review papers and three primary Zoho Creator documentation sources. The literature supports a balanced assessment: low-code can accelerate delivery, while complexity, governance and platform constraints remain design considerations [1], [2]. Zoho's documentation describes Creator

workflows and blueprints as configurable mechanisms for event handling and staged processes [3]–[5].

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Gao et al., 2026	Systematic review of low-code viability	Reports development-speed benefits with context-dependent quality and complexity outcomes.	Supports balanced evaluation; does not prove this project's performance.
2	da Silva et al., 2024	Review of low-code/no-code adoption	Reviews adoption drivers and challenges, including governance and platform dependence.	Frames platform lock-in and governance considerations.
3	Zoho Corporation, 2026	Zoho Creator Quickstart Guide	Shows an IT asset use case with forms, lookups, approvals, Blueprints and reports.	Directly relevant platform documentation.
4	Zoho Corporation, 2026	Understanding workflows	Describes workflow types and event-driven automation in Creator.	Supports terminology for form workflow and schedule.
5	Zoho Corporation, 2026	Creating blueprints	Explains stages, transitions and actions in a Blueprint.	Supports the maintenance lifecycle design.

2.5 Emerging Trends in Low-Code Development

Relevant trends include citizen development through visual builders, automation across record lifecycles, and AI-assisted application design. This project uses visual forms, reports and a dashboard together with Deluge and workflow configuration, which places it in the low-code category. No AI-generated feature or external analytics service is evidenced in the supplied export; AI and advanced analytics are therefore treated as possible extensions rather than implemented capabilities.

2.6 Proposed System

The proposed Dog Adoption Management System is a web-based application designed to simplify and improve the dog adoption process. It provides a centralized platform connecting adopters, shelters/rescuers, and administrators to manage dog profiles, user registrations, and adoption applications efficiently. Adopters can browse available dogs, submit adoption requests, and track their application status. Shelters and rescuers can add dog details, update health information, and manage adoption requests, while administrators can verify users, approve dog listings, and maintain adoption records. The system also provides automated email notifications, adoption status updates, post-adoption follow-up reminders, and adoption event management. Additionally, dashboards and reports help monitor adoption activities and improve decision-making. Overall, the proposed system reduces manual work, improves data accuracy, enhances communication, and promotes responsible dog adoption and animal welfare.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Manages and supervises the overall Dog Adoption Management System.	Verify adopter and shelter/rescuer registrations; Approve and manage dog listings.
Adopter	A user who searches for dogs and applies to adopt them.	Register and maintain personal profile details; Browse available dogs and view their health information.
Shelter/Rescuer	Manages dog information and handles adoption requests.	Register and maintain shelter/rescuer details. and maintenance updates.
Veterinary Doctor	Provides medical information and health assessments for dogs	Examine dogs and update their health records.

3.2 Functional Requirements

The requirements are expressed as observable application behavior. Test execution results are not inferred from the presence of a configured form or workflow.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Register, log in, and manage user profiles	Adopter, Shelter/Rescuer	High
FR-02	Add dog profiles and update health details	Shelter/Rescuer, Veterinary Doctor	High
FR-03	Verify users and approve dog listings	Admin	High
FR-04	Browse available dogs and submit adoption applications	Adopter	High
FR-05	Review adoption applications and update application status	Admin, Shelter/Rescuer	High
FR-06	Send email notifications and post-adoption follow-up reminders	System, Admin	Medium

FR ID	Requirement	Role(s)	Priority
FR-07	Manage adoption records, feedback, events, and generate reports	Admin, Adopter, Shelter/Rescuer	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Measure report and dashboard performance with representative data; no threshold recorded.
NFR-02	Security	Restrict access using verified Creator permissions before real employee data is used.
NFR-03	Usability	Use clear field labels and navigation; screenshots evidence a web interface.
NFR-04	Availability	Availability depends on Creator service and account; no project-specific target recorded.
NFR-05	Maintainability	Use consistent form, field and workflow names and document Deluge changes.
NFR-06	Auditability	Enable and review available record history or audit features before production; configuration not verified.

The project is implemented as a Zoho Creator application and accessed through a modern web browser. Deluge is the scripting language used by Creator. The screenshots show a web interface; no specific browser version, device specification, API testing tool, or mobile client test was recorded.

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator; edition not identified
Scripting language	Deluge
Browser	Web browser; exact browser/version not recorded
API testing tool	Not applicable; no API call supplied
Other Zoho services	No external service connection evidenced
Mobile	Not verified
Client hardware	Internet-connected computer capable of running a supported browser; minimum not measured

3.5 Use Case Model

The use case model summarizes the four user categories represented in employee records. The available evidence shows a role field on the Employee form, but does not verify that separate Creator permission profiles were configured.

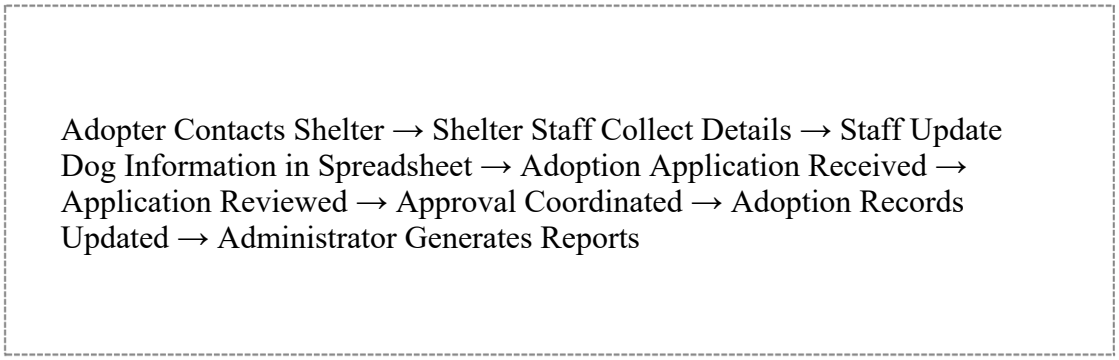


Figure 3.1: User Roles and Main Application Use Cases

Table 3.5: Use Case Description – Not evidenced

Use Case ID	UC-01
Use Case Name	Submit and track an request
Actor(s)	Adoptor
Pre-condition	User can access Creator; permissions to be verified.
Main Flow	1. Open Request. 2. Enter requester, related asset, subject, description and priority. 3. Submit. 4. Review Request Report / Statuses.
Alternate Flow	If required information is missing, correct the form before submission. Validation was not independently tested.
Post-condition	An IT Request record is available in application reports. FR-04,
Related FR	FR-06

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The application is presented through the Zoho Creator web interface. Users navigate to forms to add or update records, reports to review records, grouped views to inspect status or category, and a dashboard for summary metrics. Form data is stored in the Creator application and linked through lookups to asset, employee and request records. The exported configuration defines Deluge email actions, a schedule and Blueprint stages. No external API, CRM or analytics connection is evidenced.

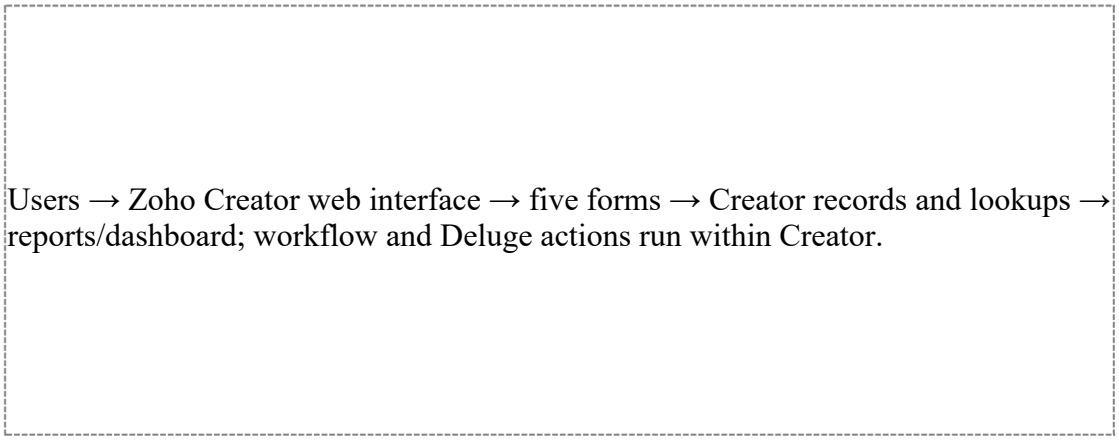


Figure 4.1: Zoho Creator Application Architecture

The system has three logical layers: (1) users access the application through the Creator web interface; (2) the application layer contains five forms, eight reports, a dashboard, workflows and Blueprint configurations; and (3) the Creator data layer stores form records and their lookup relationships. Deluge scripts and workflow actions operate within the application layer. The available export does not show a separate external integration layer.

4.2 Data Model

Relate to Unit II – Data Modelling. Present the ER diagram, then explain each entity and its relationships. In Zoho Creator, relationships are realised through lookup fields (one-to-many / many-to-many) and subforms (parent–child).

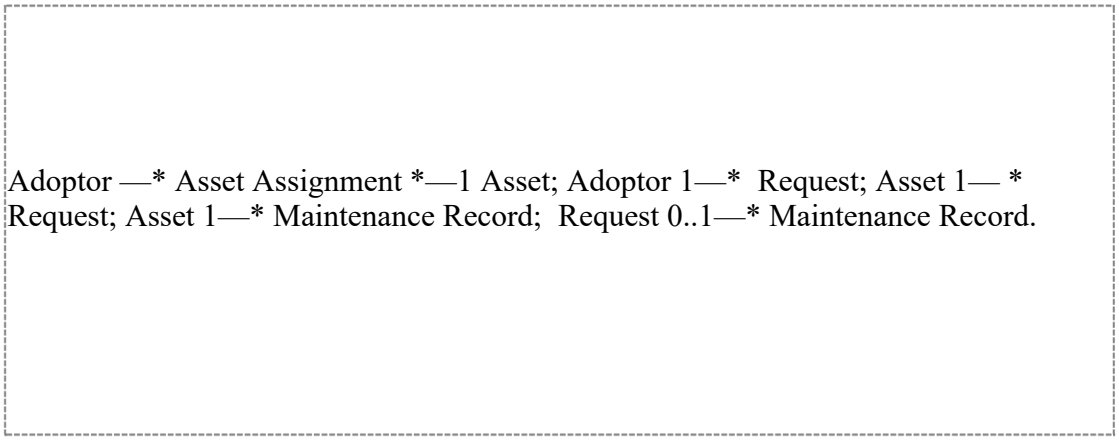


Figure 4.2: Core Entity Relationships in the Application

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Asset	Asset Id, tag, name, category, cost, warranty end, status	Assignment, Request, Maintenance	One asset to multiple records	Asset lookup/list fields
Adopter	Employee Id, name, department, email, role, manager	Assignment, IT Request	Employee linked to assignments and requests	Employee lookup/list fields
Asset Assignment	Assignment Id, asset, employee, dates, status	Asset, Employee	Many assignments may reference asset or employee	Asset and Employee list fields

4.3 Form Design

The project contains five forms. Asset records capture identification, category, manufacturer, model, serial number, purchase and warranty dates, cost, assignees, location and notes. Employee records capture identity, department, contact, role, active status, location, manager and department code. Asset Assignment records link assets and employees, with assignment and return details. IT Request records link requester and related asset and track priority, status, assigned staff and resolution. Maintenance Record links an asset and optionally an IT request, and records type, description, technician, status, cost and dates. The following table gives representative field specifications; the complete field set is in the supplied Creator export.

Table 4.2: Field Specification – Not evidenced Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Asset Id	asset_id	Single Line / text	Not specified	No validation evidenced	Asset record identifier
Category	category	Drop Down / picklist	Not specified	Laptop, Monitor, Accessory, Desktop, Printer, Mobile; other allowed	Asset classification
Purchase Cost	purchase_cost	Currency (USD)	Not specified	No default evidenced	Shown in cost reports
Warranty End Date	warranty_end_date	Date	Not specified	Date field; used by schedule	Reminder criteria in export
Status	status	Drop Down / picklist	Not specified	Assigned, Retired, Available, Under Maintenance	Asset state

The exported form definitions provide the field names and types summarized in Table 4.2. The screenshots in Chapter 5 show the live forms and reports.

4.4 Process and Workflow Design

Draw a flowchart or swim-lane diagram for each automated process (e.g., request → validation → approval → notification → status update). Mark where Deluge scripts and workflows run.

Adopter Contacts Shelter → Shelter Staff Collect Details → Staff Update Dog Information in Spreadsheet → Adoption Application Received → Application Reviewed → Approval Coordinated → Adoption Records Updated → Administrator

Figure 4.3: Asset Return Notification and Warranty Reminder Processes

4.5 User Interface and Navigation Design

> Show the navigation structure (sections/menus) and wireframes of key pages and dashboards before implementation.

Dashboard; IT Request (form/report/statuses); Asset Assignments (form/report); Assets (form/all assets/categories); Employees (form/all employees/departments); Maintenance Records (form/all records/types).

Figure 4.4: Application Navigation and Dashboard

4.6 Access Control Design

> The employee form includes an organizational role value (Administrator, Employee, Manager or IT Staff). This records a business role, but it is not evidence of Creator permission-profile or record-level access configuration. The matrix below is a proposed access design for a future role-permission review; permissions must be verified in Creator before being treated as implemented.

Table 4.3: Role–Permission Matrix

Component	Administrator	Manager	IT Staff	Employee
Asset and Employee forms	Proposed: A E V D	Proposed: V	Proposed: A E V	Proposed: V
Assignment and Request forms	Proposed: A E V D	Proposed: V	Proposed: A E V	Proposed: A V (own)
Maintenance forms and reports	Proposed: A E V D	Proposed: V	Proposed: A E V	Proposed: —
Dashboard and grouped reports	Proposed: V	Proposed: V	Proposed: V	Proposed: —
Status	Not verified in supplied evidence	Not verified	Not verified	Not verified

CHAPTER 5
IMPLEMENTATION

This chapter reports the configuration visible in the supplied .ds export and screenshots. Where the material does not demonstrate a feature, the report identifies that limit instead of assuming it was configured.

5.1 Application Setup

The application is named Internal IT Asset Management System and uses the link name Internal_IT_Asset_Management_System. The export defines five forms and groups application navigation into Dashboard, IT Request, Asset Assignments, Assets, Employees and Maintenance Records. The supplied screenshots show the live interface and sample records. The exact creation method and environment promotion history are not present in the export.

Not evidenced

Dashboard Overview

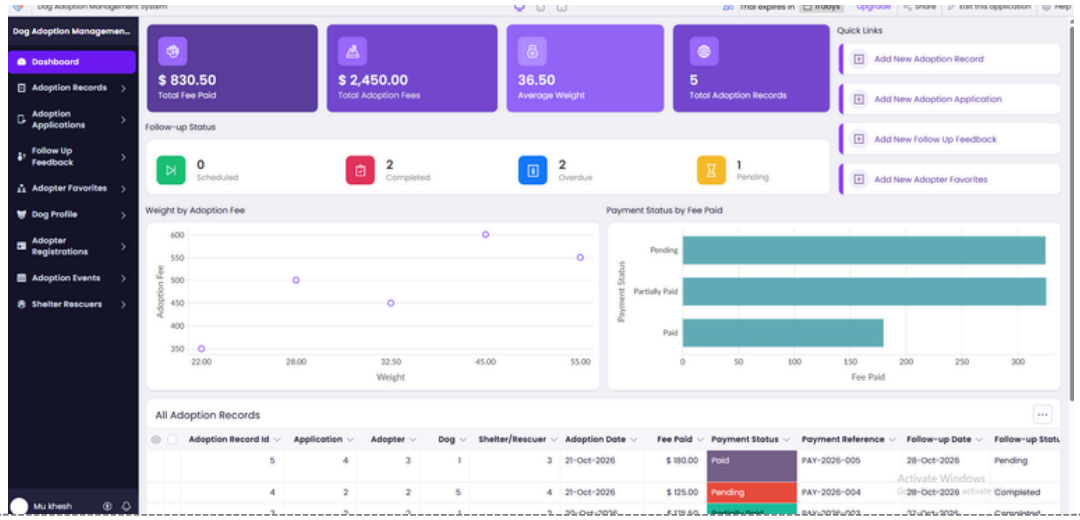


Figure 5.1: Application Dashboard and Navigation

5.2 Forms and Data Capture

The five forms capture the core data. Asset and Employee are master records. Asset Assignment links an asset to one or more employees and stores assignment status and dates. IT Request links the requester and related asset and captures issue details, priority and resolution. Maintenance Record links an asset and request and records maintenance type, technician, status, cost and completion. The exported .ds file defines text, picklist, date, currency, list/lookup, name, email, checkbox, phone, date-time and textarea fields. The screenshots show the visible form layouts and sample register data.

Not evidenced

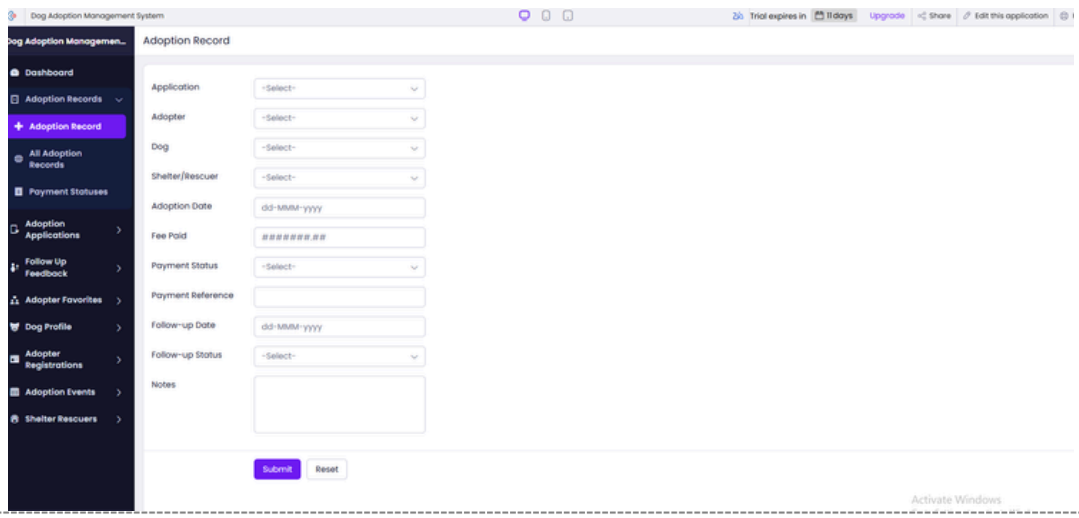


Figure 5.2: Add Adoption Request Form

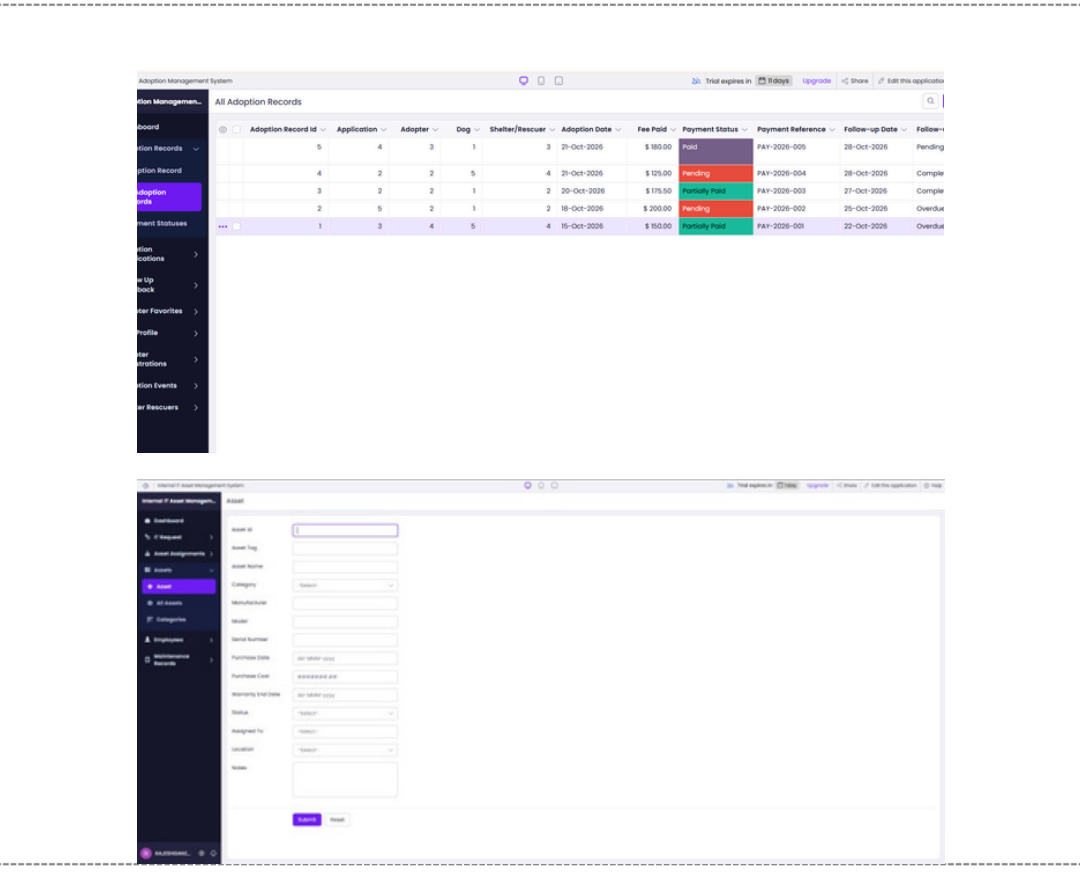


Figure 5.3: Adoption Records

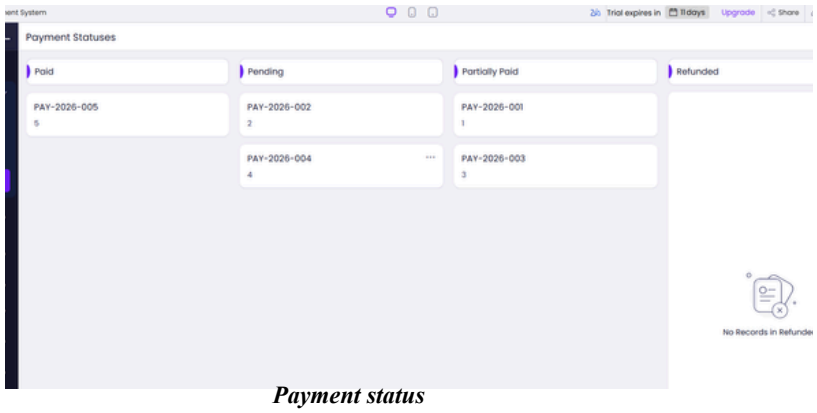
5.3 Reports

The application defines eight reports: IT Request Report, IT Request by Status, Asset Assignment Report, Asset Report, Asset by Category, Employee Report, Employee by Department, Maintenance Record Report and Maintenance Record by

Maintenance Type. The screenshots show list reports, category and department boards, and a status board. The dashboard displays purchase-cost summaries, request and assignment counts, request priority counts, and asset purchase cost by status and category. These are descriptive views over the sample records; no performance benchmark or custom action is evidenced.

Table 5.1: Reports Implemented in the Application

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
IT Request Report	List	IT Request	Status and priority shown	None evidenced	Access not verified
IT Request by Status	Kanban / grouped	IT Request	Grouped by status	Status board shown	Access not verified
Asset Assignment Report	List	Asset Assignment	Assignment date and status shown	None evidenced	Access not verified
Asset Report / Asset by Category	List / grouped	Asset	Status, category and purchase cost	Cost/category chart shown	Access not verified
Employee / Department / Maintenance reports	List / grouped	Employee / Maintenance	Grouped by department or type	None evidenced	Access not verified



Payment status

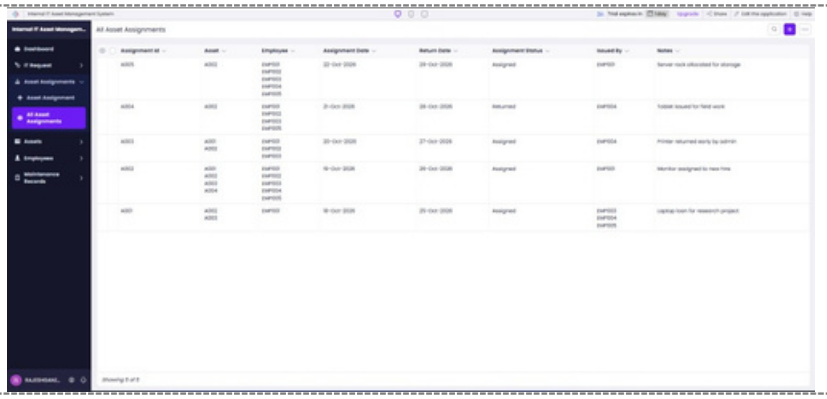


Figure 5.4: Request and Assignment Reports

5.4 Pages and Dashboards

> The Dashboard page combines four summary panels, request-priority counts, charts for asset costs by status and category, quick links, and an IT Request Report. The page gives administrators and IT staff a high-level view of the sample data. The exact permission audience is not established by the supplied evidence.

Not evidenced

Asset Inventory

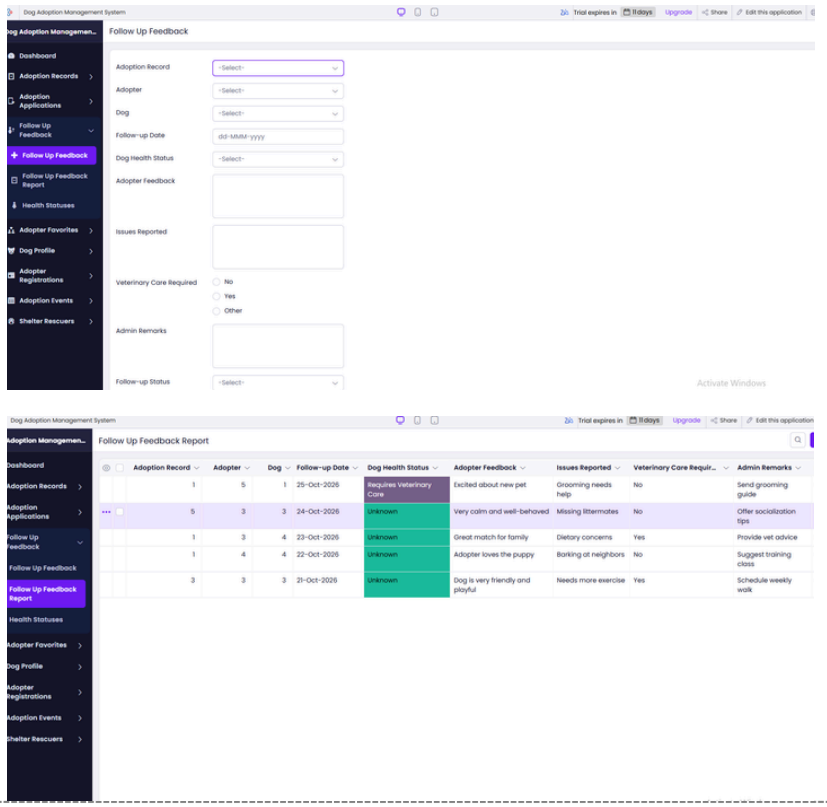


Figure 5.5: Dashboard and Asset Categories

5.5 Deluge Scripting

The exported application includes Deluge scripts in two automations. The asset-assignment form workflow checks whether an edited assignment has Returned status, retrieves the related employee and manager, and emails the manager when a manager email is present. The warranty schedule selects assets whose warranty end date falls within the next 30 days and emails assigned employees, falling back to the logged-in user when no employee email is available. The code excerpts summarize the exported logic; no separate validation script or API integration was found.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Asset Assignment → On Edit → On Success	If status is Returned, retrieve employee, manager and asset, then email manager if address is present.	Record criteria, related-record fetch, condition, sendmail
DS-02	Asset → Warranty End Date schedule at 10:00 Asia/Kolkata	Find assets with warranty ending in next 30 days and email assigned employees; use login user as fallback.	Date criteria, loop, list collection, sendmail
DS-03	No separate function evidenced	No standalone function identified in supplied export.	Not applicable
Not evidenced	Not evidenced	Not evidenced	Not evidenced
Not evidenced	Not evidenced	Not evidenced	Not evidenced

Code 5.1: Notify the manager after an asset return

```
// DS-01 | Asset Assignment | On Edit: On Success
assetAssignment = asset_assignment[assignment_status == "Returned" && ID ==
input.ID];
if(assetAssignment.count() > 0)
{
 employeeRecord = employee[ID == assetAssignment.employee.ID];
 managerEmail = employeeRecord.manager;
 if(!managerEmail.isNull() && !managerEmail.isEmpty())
 {
  assetRecord = asset[ID == assetAssignment.asset.ID];
  subject = "Asset Returned: " + assetRecord.asset_name;
  message = "<p>An asset has been returned.</p>";
  message = message.concat("<p>Asset: " + assetRecord.asset_name + "</p>");
  sendmail [ from :zoho.loginuserid to :managerEmail subject :subject message
:message content type :HTML ];
 }
}
```

Code 5.2: Send a warranty expiry reminder

```
// DS-02 | Asset | Warranty End Date schedule at 10:00 Asia/Kolkata
warrantyExpiringAssets = asset[warranty_end_date in next 30 days];
for each assetRecord in warrantyExpiringAssets
{
 assignedEmployeeEmails = List:String();
 if(!assetRecord.assigned_to.isNull())
 {
  employeeRecords = employee[ID in assetRecord.assigned_to];
  for each employeeRecord in employeeRecords
  {
   if(!employeeRecord.email.isNull())
assignedEmployeeEmails.add(employeeRecord.email);
  }
 }
 recipientEmails = zoho.loginuserid;
 if(assignedEmployeeEmails.size() > 0) recipientEmails =
assignedEmployeeEmails.toString(",");
 sendmail [ from :zoho.loginuserid to :recipientEmails subject :"Warranty Expiry
Alert - " + assetRecord.asset_name message :"Warranty ends on " +
assetRecord.warranty_end_date.toString("dd MMM yyyy") ];
}
```

The first script runs after an Asset Assignment edit and sends a message only when the assignment is marked Returned and a manager email is available. The second script is attached to the warranty date schedule and checks the next-30-days window before sending a reminder. The supplied screenshots show the workflow and schedule enabled.

They do not include execution logs or delivery receipts, so runtime success is not independently verified.

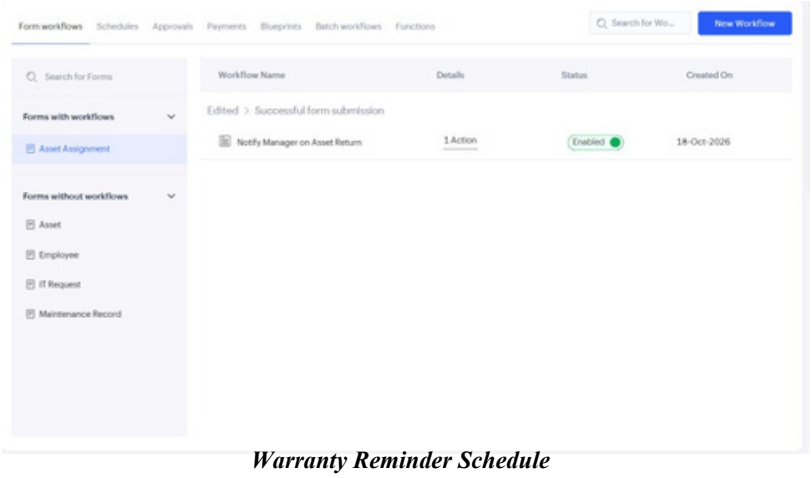
5.6 Workflows and Automation

The export defines a form workflow named “Notify Manager on Asset Return,” triggered after editing an Asset Assignment. It also defines the “Warranty Expiry Reminder” schedule on Asset, based on Warranty End Date at 10:00 in Asia/Kolkata. The export includes an Asset Maintenance Lifecycle blueprint and stage definitions for IT Requests. Screenshots show the form workflow, schedule and maintenance blueprint enabled. No separate approval workflow is evidenced.

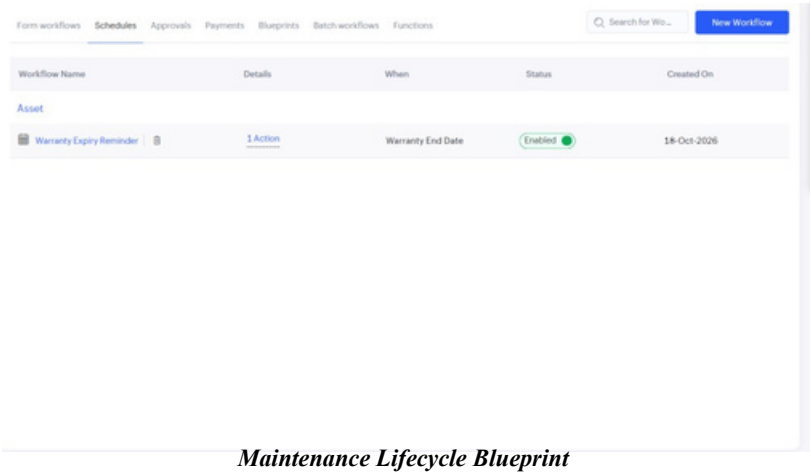
Table 5.3: Workflows and Blueprint Processes

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Notify Manager on Asset Return	Form workflow	Asset Assignment edited; On Success	Status = Returned; manager email present	Email asset, employee, return date and notes	DS-01
Warranty Expiry Reminder	Schedule	Warranty End Date at 10:00 Asia/Kolkata	Warranty date in next 30 days	Email assigned employee addresses; fallback to logged-in user	DS-02
Request lifecycle	Blueprint stage configuration	Request record lifecycle	Stages defined in export	Submission, manager approval, IT assignment, progress and closure stages	Blueprint stages
Asset Maintenance Lifecycle	Blueprint	Maintenance Record lifecycle	Starts at Request Created	Review, approval, parts, technician assignment and maintenance progress	Blueprint
Not evidenced	Not evidenced	Not evidenced	Not evidenced	Not evidenced	Not evidenced

Asset Return Workflow



Warranty Reminder Schedule



Maintenance Lifecycle Blueprint

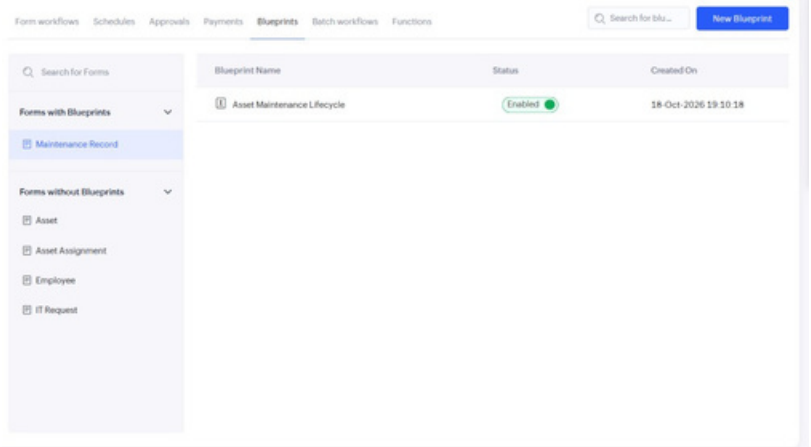


Figure 5.6: Request Status, Employee and Workflow Configuration

5.7 Integrations

> No external service integration, Creator REST API call, invokeurl call or Postman response is present in the supplied export. The workflow uses Creator's built-in sendmail action. Zoho Mail, CRM, Sheets, Catalyst and external APIs are not reported as integrated.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Built-in email action	Zoho Creator sendmail	Workflow and schedule actions	Creator logged-in identity	Manager return notice and warranty reminder
External integration	None evidenced	None evidenced	Not applicable	No external exchange in supplied export
Creator API / webhook	None evidenced	None evidenced	Not applicable	No API or webhook supplied

Code 5.3: No external integration script is included in the supplied export.

```
No external integration script is included in the supplied project export.
```



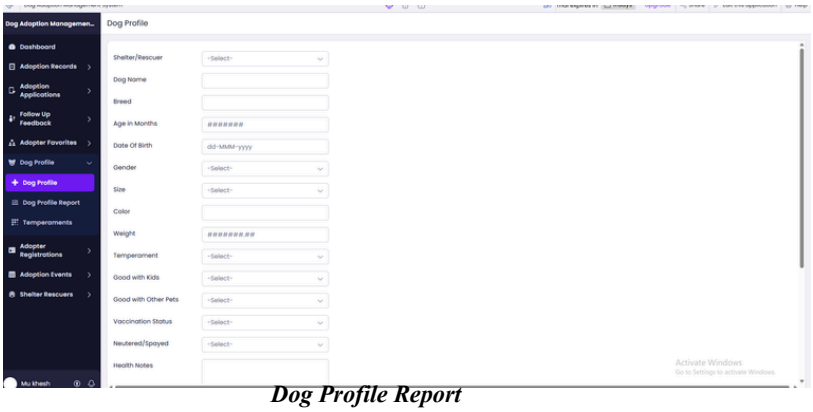
Figure 5.7: No External Integration Configuration Supplied

5.8 User Management and Role-Based Access

> The Employee form stores a role value and manager name, while the application screens show signed-in access to the Creator application. The supplied export and screenshots do not show user invitations, permission profiles, record-level rules, portal configuration or field-level restrictions. Table 4.3 therefore records a recommended permission matrix for verification, not a claim that it is already enforced.

Not evidenced

Dog Profile



Dog Profile Report

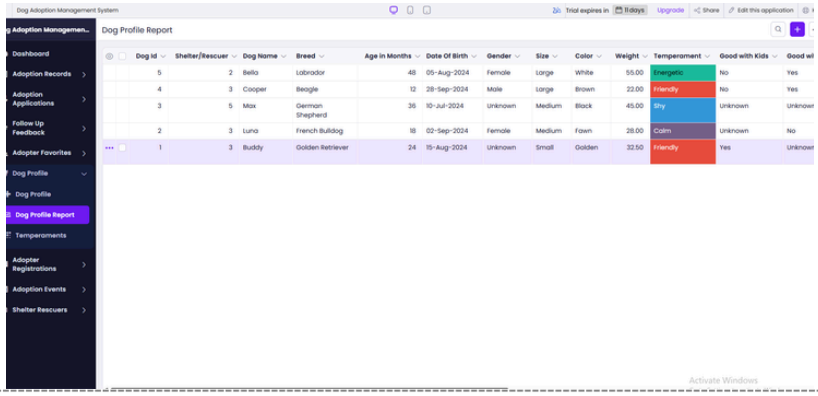


Figure 5.8: Dog Profile And Dog Profile Report

5.9 AI and Analytics Features

> The dashboard includes count and cost summaries and bar charts for purchase cost by asset status and category. The supplied export does not evidence Zia, AI-generated fields, prediction, OCR, sentiment analysis or a Zoho Analytics connection. The analytics discussion is limited to the Creator dashboard and reports shown in the screenshots.

Not evidenced

Dashboard Charts

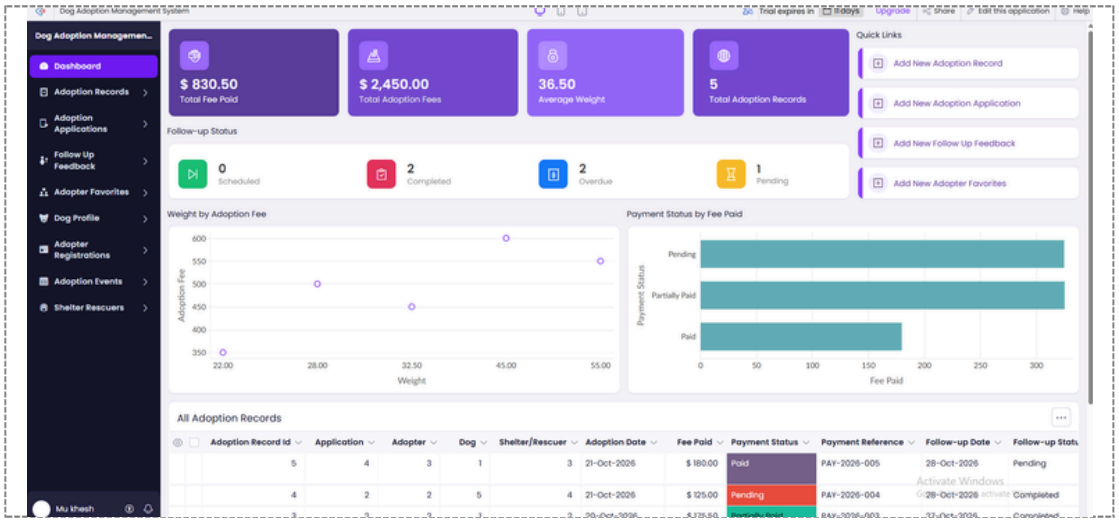


Figure 5.9: Dashboard Charts and Cost Summaries

5.10 Pro-Code Extension

> No pro-code extension, Catalyst service, custom JavaScript widget or Creator REST API endpoint is evidenced in the export. For this project's current scope, Creator forms, reports, Deluge, workflow and blueprint configuration provide the demonstrated functions. A separate service could be considered if future requirements exceed Creator's scripting or integration capabilities.

Not evidenced

5.11 Mobile and Portal Access

> The supplied screenshots document the web application. Mobile layout, offline use and customer portal access were not tested or documented for this project.

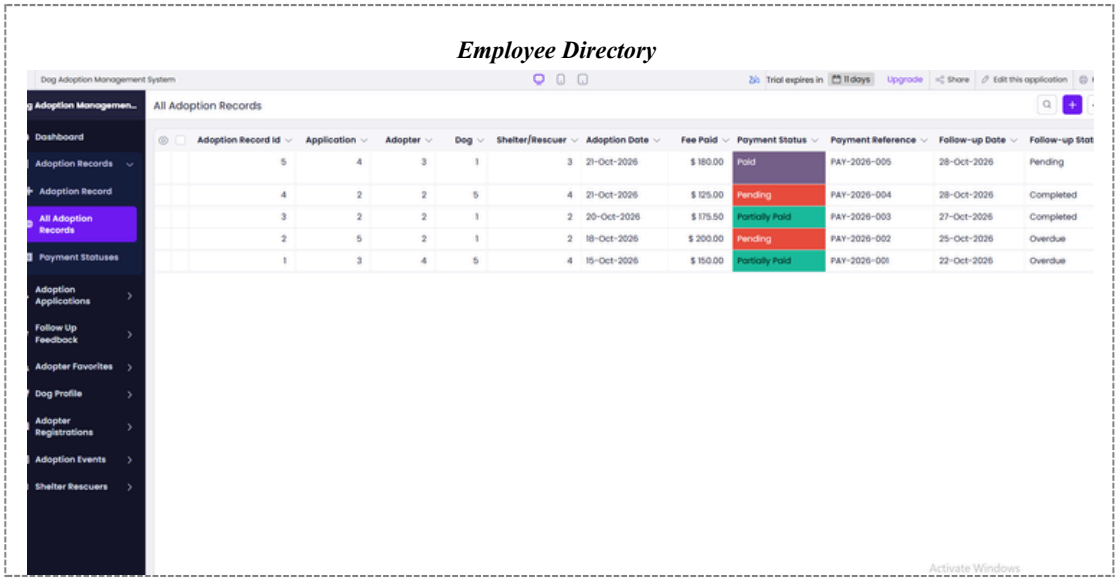


Figure 5.10: Employee Directory; Mobile and Portal Access Not Verified

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Verification should cover required and invalid form inputs, lookup relationships, report content, workflow triggers, schedule criteria, Blueprint transitions and access permissions. The supplied screenshots confirm that sample data appears in the reports and dashboard, and that the workflow, schedule and blueprint are shown as enabled. They do not constitute a complete test log. The cases in Table 6.1 are a verification plan; results remain "Not recorded" unless a screenshot or export directly demonstrates the result.

Not evidenced

6.2 Test Cases

The following test cases are derived from the functional requirements. The provided materials do not include a completed execution log, so actual results and pass status are not invented.

Table 6.1: Functional and Workflow Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Create an asset record	Enter asset identity, category and status; submit.	Asset appears in Asset Report.	Sample records visible in screenshot; no execution log.	Evidence shown
TC-02	FR-02	Create an employee record	Enter employee ID, name, department and role; submit.	Employee appears in Employee Report.	Sample rows visible in screenshot.	Evidence shown
TC-03	FR-03	Record an assignment	Select asset and employee; enter date and status; submit.	Assignment appears in report.	Sample rows visible in screenshot.	Evidence shown

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-04	FR-04	Review request status board	Open IT Request by Status report.	Requests appear in status columns.	Sample cards visible in screenshot.	Evidence shown
TC-05	FR-05	Review maintenance records	Open Maintenance Record report.	Records show type, asset, status and cost.	Sample records visible in screenshot.	Evidence shown

6.3 Debugging Deluge Scripts

> No Deluge debug output or defect history was included. A repeatable debugging approach is to inspect form-workflow criteria, verify related employee and asset lookups, confirm manager email is present, review the workflow execution log, and test schedule dates inside and outside the 30-day window. A defect and fix are not claimed without a recorded example.

Not evidenced

Request Status Board

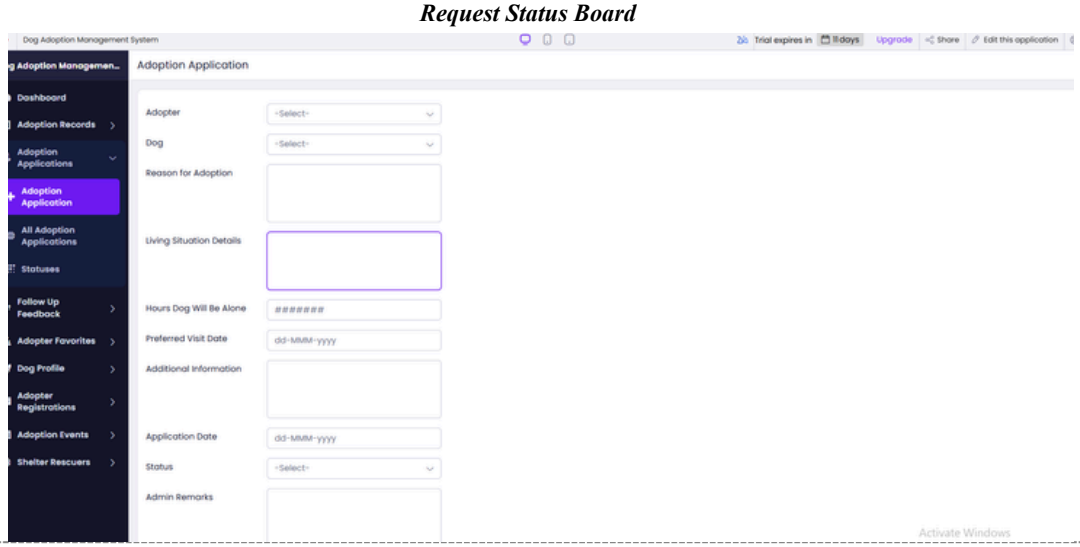


Figure 6.1: Request Status Board; No Debug Log Supplied

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
Not recorded	No defect log supplied.	Not available	Capture issues and fixes during testing.	Not documented

Bug ID	Description	Root Cause	Fix Applied	Status

6.5 User Acceptance Testing

> No user acceptance feedback was supplied. Record this section after two or more users complete role-specific tasks and provide feedback.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Not recorded	Not recorded	No UAT feedback supplied.	Conduct and document UAT before deployment.

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

> The exported application and live screenshots demonstrate a working Creator application interface. No Development-to-Stage-to-Production promotion record, published release date, rollback plan or environment configuration was supplied. The checklist marks such items as unverified. Before production use, test the app in a non-production environment, confirm permissions and email recipients, and record the release and rollback steps.

Not evidenced

Live Application Dashboard

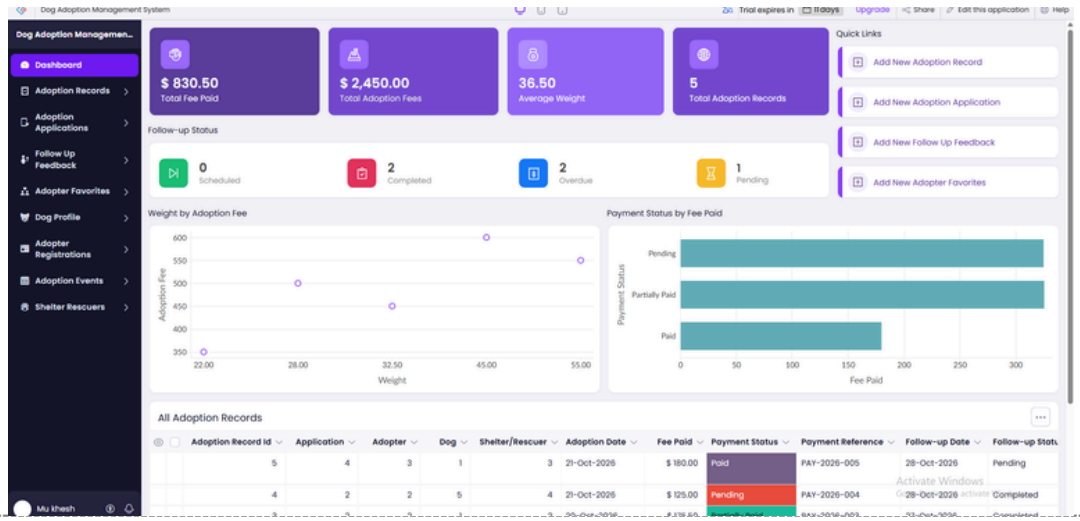


Figure 7.1: Live Dashboard; Environment History Not Supplied

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Test cases executed in Development / Stage	Not evidenced
2	Sample/test data removed or isolated	No; screenshots show demonstration records
3	Roles, permissions and users configured in Production	Not evidenced
4	Connections authorized in Production	Not applicable / not evidenced

S.No.	Checklist Item	Done (✓)
5	Schedules and workflows enabled	Enabled status shown; execution not verified
6	Application published and URL verified	Live app screenshots supplied; deployment history not documented
7	Version / release notes recorded	Not evidenced

7.2 Performance Optimisation

No before-and-after timing measurements or optimization log were supplied. The application provides grouped views and a dashboard over a small sample dataset, but the report does not claim response-time improvements. Performance should be measured with representative record volumes before deployment.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
No measured optimization supplied	Dashboard and reports	Not measured	Not measured	No performance gain claimed
No benchmark supplied	Application data volume	Sample records shown	Production volume unknown	Measure before release

7.3 Security Features

The employee form has a role field, and email actions use the Creator login identity. The provided project files do not verify configured permission profiles, record-level rules, encrypted fields, IP restrictions, multi-factor requirements or audit settings. These must be reviewed in Creator before the application is used with real employee or asset data.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Creator role/profile permissions	Not verified	Review access before use
Sensitive field encryption	Not evidenced	Assess protection of personal and asset data
Workflow sender identity	sendmail uses zoho.loginuserid	Avoid hard-coded sender credentials; verify configuration
Audit and record history	Not verified	Enable/review for accountability

7.4 Governance and Administration

The application uses named forms, reports and workflow components visible in the export. The supplied files do not show backup schedules, change approvals, audit-trail settings, license management or production ownership. These items should be assigned and documented as part of deployment governance.

Not evidenced

No audit-trail or administration screenshot was supplied. The maintenance Blueprint is shown in Figure 5.6.

Figure 7.2: Audit Trail Not Verified; Blueprint Shown in Figure 5.6

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The screenshots illustrate the end-to-end data views. An IT team can register assets and employees, record an assignment, submit an IT request, follow requests through status stages, and create maintenance records linked to assets. The dashboard consolidates sample purchase costs and counts. The examples are demonstration records and are not measured pilot results.

The dashboard displays headline metrics and charts alongside the request report and quick links.

Scenario Step 1 Dashboard

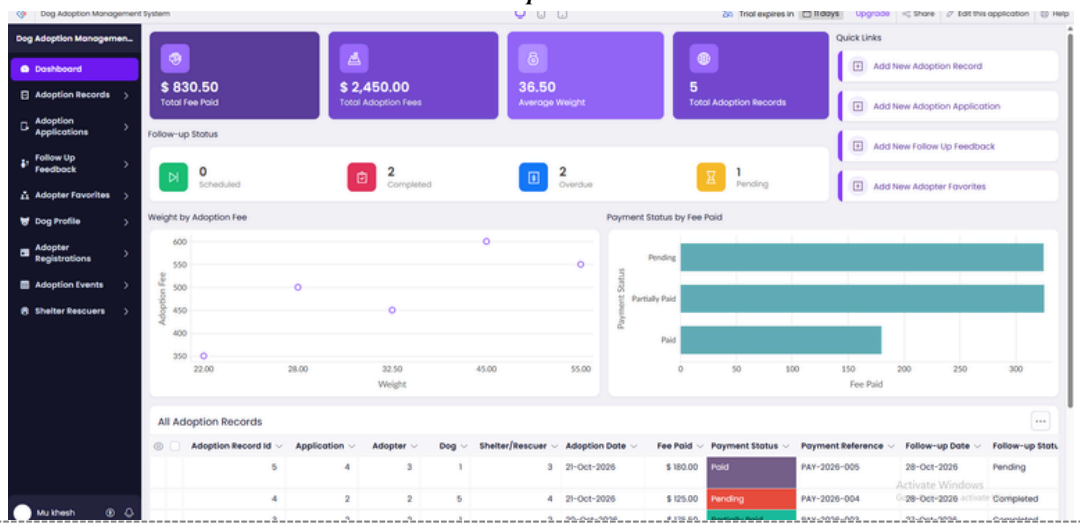
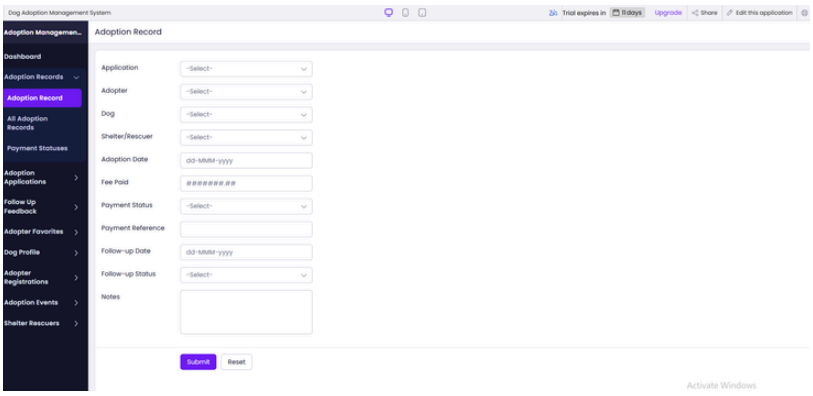


Figure 8.1: Maintenance Record Form

Scenario Step 2 Adoption Request



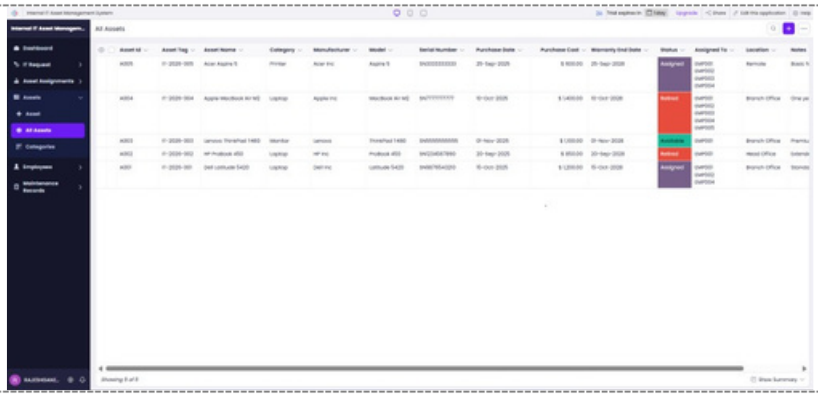


Figure 8.2: Workflow Configuration

Scenario Step 3 Maintenance Records



Maintenance Type View

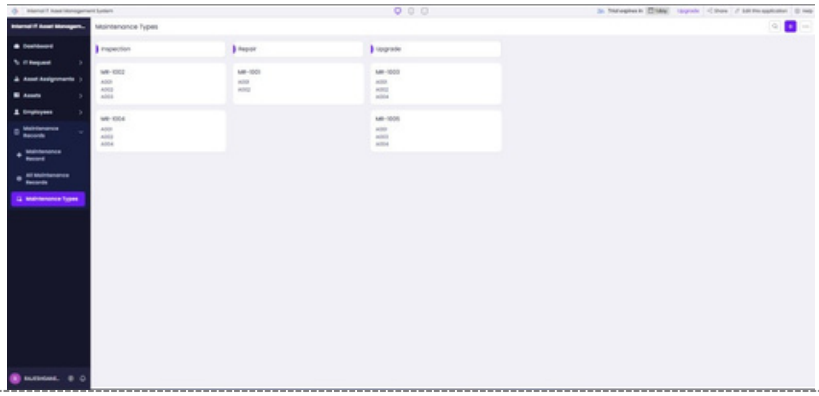


Figure 8.3: Maintenance Type Board

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Create structured operational records	Five Creator forms	Five form types confirmed in export and screenshots.
Represent process status and automation	Form workflow, schedule and Blueprint stages	Configuration present in export; enabled components shown.

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Provide reports and summaries	Eight reports and Dashboard page	Reports and dashboard visible with sample data.

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Record access	Separate files assumed for baseline	Related Creator forms and reports	Centralized in demonstrated app; quantitative gain not measured
Manual steps	Manual follow-up assumed for baseline	Return notification and warranty schedule	Automation configured; execution not verified
Report generation	Manual summary assumed for baseline	Eight reports and dashboard	Views preconfigured; time saved not measured
Data accessibility	Distributed records assumed	Creator web interface	Shared interface demonstrated; role access not verified

8.4 Limitations of the Low-Code Approach

> The project depends on Zoho Creator's data model, report types, workflow configuration and Deluge runtime. This can make the application faster to configure for the current forms and reports, but it also ties the implementation to Creator concepts and services. Complex interfaces, very high-volume processing, advanced integration orchestration and custom analytics may require external services or pro-code extensions. The project files do not document a platform limit encountered during implementation, so these are design considerations rather than observed defects.

Not evidenced

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

> The forms, reports, dashboard layout, picklist options and process stages are primarily visual configuration. The manager notification and warranty reminder require low-code Deluge logic. A custom mobile client, advanced API integration or high-volume analytics service would move selected components toward pro-code

> development. The appropriate boundary depends on future requirements for scale, external connectivity, portability and specialized user experience.

Not evidenced

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Dog Adoption Management System is designed to simplify and improve the dog adoption process by providing a centralized platform for adopters, shelters/rescuers, administrators, and veterinary doctors. The system helps manage dog profiles, user registrations, adoption applications, medical records, and adoption details efficiently. It also provides automated notifications, follow-up reminders, feedback management, and adoption event tracking. By reducing manual work and maintaining accurate records, the system improves communication, saves time, and ensures better coordination among users. Overall, the project promotes responsible dog adoption, supports animal welfare, and helps dogs find safe and loving homes.

Not evidenced

9.2 Future Enhancements

- Configure and verify Creator permission profiles and record-level access for
- Complete a test log and user acceptance review, including negative tests and
- Evaluate integrations, mobile access and predictive warranty or maintenance

9.3 Learning Outcomes and Career Relevance

The project provided practical experience with low-code form modeling, lookup relationships, reports, dashboards, Deluge scripting and workflow stages. These skills are relevant to low-code application development, business analysis and workflow automation roles. Further work on access control, testing, deployment governance and integration design would strengthen the application for operational use.

Not evidenced

REFERENCES

References are listed in IEEE style and cited in the report.

[1] D. Gao, F. Fagerholm, and V. Toivanen, “What does current research say about the viability of low-code development? A systematic literature review,” Journal of Systems and Software, vol. 239, art. 112893, 2026, doi: 10.1016/j.jss.2026.112893.

[2] M. O. Ajimati, N. Carroll, and M. Maher, “Adoption of low-code and no-code development: A systematic literature review and future research agenda,” Journal of Systems and Software, vol. 222, art. 112300, 2025, doi: 10.1016/j.jss.2024.112300.

[3] Zoho Corporation, “A Quickstart Guide to Zoho Creator,” Zoho Creator Help. [Online]. Available: https://www.zoho.com/creator/help/new-quickstart-guide.html. Accessed: 19 Oct. 2026.

[4] Zoho Corporation, “Understanding workflows,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows. Accessed: 19 Oct. 2026.

[5] Zoho Corporation, “Creating blueprints,” Zoho Creator Help. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/blueprints/articles/create-a-blueprint. Accessed: 19 Oct. 2026.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

This checklist distinguishes features evidenced by the supplied .ds export and screenshots from features not verified in those materials.

Table A.1: Feature Coverage Checklist

Unit	*Zoho Creator Feature*	*Done*	*Where inthe Application*	*Report Section*	*Figure / Code No.*
I	Application created & sections/menu organised	Yes	Dashboardandsix menu groups	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Yes	Five forms; multiple field types	5.2	Tables 4.2, Fig. 5.2–5.3
II	Lookup field (relationship between forms)	Yes	Asset, Employee and Request list fields	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	No evidence	Not found in supplied export/screenshots	4.2, 5.2	—
II	Field validations / layout rules	No evidence	Not found in supplied export/screenshots	5.2	—
II	List report with filter, grouping, custom action	No evidence	Not found in supplied export/screenshots	5.3	—
II	Kanban / Calendar / Timeline / Map report	No evidence	Not found in supplied export/screenshots	5.3	—
II	Pivot table / chart report	No evidence	Not found in supplied export/screenshots	5.3	—
II	Page / dashboard built with UI Builder	Yes	Dashboard page	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	No evidence	Not found in supplied export/screenshots	5.5	—
II	Deluge – functions, fetch, loops, collections	No evidence	Not found in supplied export/screenshots	5.5	—

Unit	*Zoho Creator Feature*	*Done*	*Where in the Application*	*Report Section*	*Figure / Code No.*
II	Email / in-app notifications	Partial	Return noticeand warranty reminder configured	5.5, 5.6	DS-01, DS-02
II	Form workflow	Yes	Notify Manager on Asset Return	5.6	Fig. 5.6
II	Scheduled workflow	Yes	Warranty Expiry Reminder	5.6	Fig. 5.6
II	Approval process	No evidence	Not found in supplied export/screenshots	5.6	—
II	Blueprint	Yes	Maintenance lifecycle and IT Request stages	5.6	Fig. 5.6
II	Batch / report workflow	No evidence	Not found in supplied export/screenshots	5.6	—
II	Testing & debugging evidence	Partial	Screenshots show sample outputs; no test log	6	Table 6.1
III	Integration with a Zoho service	No evidence	Not found in supplied export/screenshots	5.7	—
III	Connection / REST API call (invokeurl)	No evidence	Not found in supplied export/screenshots	5.7	—
III	Webhook or Creator REST API	No evidence	Not found in supplied export/screenshots	5.7	—
III	Users, roles and permission profiles	No evidence	Not found in supplied export/screenshots	5.8	—
III	Record-level access criteria / customer portal	No evidence	Not found in supplied export/screenshots	5.8	—
III	Zia / AI feature	Partial	Role values shown; permission profiles unverified	5.9	Fig. 5.8

Unit	*Zoho Creator Feature*	*Done*	*Where in the Application*	*Report Section*	*Figure / Code No.*
III	Analytics (charts / Zoho Analytics)	No evidence	Not found in supplied export/screenshots	5.9	—
IV	Performance optimisation applied & measured	No evidence	Not found in supplied export/screenshots	7.2	—
IV	Deployment across environments	Partial	Creator charts shown; no Zoho Analytics	7.1	Fig. 5.5
IV	Security controls (encryption, OAuth, permissions)	No evidence	Not found in supplied export/screenshots	7.3	—
IV	Governance (audit trail, administration)	No evidence	Environment promotion not documented	7.4	—
V	Pro-code extension (Catalyst / widgets / custom functions)	No evidence	Not found in supplied export/screenshots	5.10	—
V	Low-code limitations & spectrum analysis	No evidence	Not found in supplied export/screenshots	8.4, 8.5	—
—	Mobile app / portal access	No evidence	Not found in supplied export/screenshots	5.11	—

Total evidenced features: 12 / 32; remaining items are not evidenced or are not applicable to the supplied implementation.

APPENDIX B – COURSE OUTCOME MAPPING

Course outcome evidence is mapped to the report sections listed in the table. Self-assessment should be reviewed by the student and examiner.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain low-code development principles and value	Ch. 1, 2	Review by student	
CO2	Develop forms, reports, workflows and Deluge scripts	Ch. 4, 5, Appendix C	Review by student	
CO3	Integrate services and apply role-based security	Ch. 4.6, 5.7–5.8; gaps identified	Review by student	
CO4	Deploy, secure and govern Creator applications	Ch. 7; requirements and evidence gaps	Review by student	
CO5	Evaluate low-code, AI-assisted development and pro-code transition	Ch. 5.9–5.10, 8.4–8.5	Review by student	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

> The exported source establishes the workflow and schedule logic described in Chapter 5. Complete source snippets are reproduced below where identifiable in the supplied export.

C.1 Asset Assignment Workflow – Notify Manager on Asset Return

```
// DS-01 | Asset Assignment | On Edit: On Success
assetAssignment = asset_assignment[assignment_status == "Returned" && ID ==
input.ID];
if(assetAssignment.count() > 0)
{
 employeeRecord = employee[ID == assetAssignment.employee.ID];
 managerEmail = employeeRecord.manager;
 if(!managerEmail.isNull() && !managerEmail.isEmpty())
 {
  assetRecord = asset[ID == assetAssignment.asset.ID];
  subject = "Asset Returned: " + assetRecord.asset_name;
  message = "<p>An asset has been returned.</p>";
  message = message.concat("<p>Asset: " + assetRecord.asset_name + "</p>");
  sendmail [ from :zoho.loginuserid to :managerEmail subject :subject message
:message content type :HTML ];
 }
}
```

C.2 Asset Warranty Expiry Reminder Schedule

```
// DS-02 | Asset | Warranty End Date schedule at 10:00 Asia/Kolkata
warrantyExpiringAssets = asset[warranty_end_date in next 30 days];
for each assetRecord in warrantyExpiringAssets
{
 assignedEmployeeEmails = List:String();
 if(!assetRecord.assigned_to.isNull())
 {
  employeeRecords = employee[ID in assetRecord.assigned_to];
  for each employeeRecord in employeeRecords
  {
   if(!employeeRecord.email.isNull())
assignedEmployeeEmails.add(employeeRecord.email);
  }
 }
 recipientEmails = zoho.loginuserid;
 if(assignedEmployeeEmails.size() > 0) recipientEmails =
assignedEmployeeEmails.toString(",");
 sendmail [ from :zoho.loginuserid to :recipientEmails subject :"Warranty Expiry
Alert - " + assetRecord.asset_name message :"Warranty ends on " +
assetRecord.warranty_end_date.toString("dd MMM yyyy") ];
}
```

APPENDIX D – USER MANUAL

> This user guide describes the main web tasks visible in the supplied application. Access depends on a Creator user account and permissions assigned by the application owner.

D.1 Accessing the Application

Open the Dog Adoption Management System in Zoho Creator using the URL provided by the application owner. Sign in with your assigned account. Use the left navigation to open Dashboard, Request, Asset Assignments, Assets, Maintenance Records. The screenshots document web access; mobile and portal steps are not verified.

D.2 Instructions for Adoptoer

- 1. Open Assets and choose Asset to register equipment. Enter its asset tag, name, category, manufacturer, dates, cost, status, location and notes, then submit.
- 2. Open Asset Assignment to record the asset, employee, assignment date and issuer. Use the Asset Assignment report to review assignment and return status.

D.3 Instructions for Adoptor

- 1. Open Request and enter request type, requester, related asset, subject, description, priority and submission details. Submit the form and refer to the request report for status.
- 2. Managers can review requests and assignment information using the reports. Approval permissions and notification delivery depend on configuration not verified in the supplied materials.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	Mukhesh L	Register No.	2116241501121
Project Title	Internal IT Asset Management System	Date	26 October 2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	***Criterion***	***Indicators***	***Max. Marks***	***Marks Awarded***
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		***100***	

Table E.2: Performance Level Descriptors

Level	*% of Max. Marks*	*Descriptor*
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner ***Signature of External Examiner***

Name: _____ Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	RajeshGanesh S	Register No.	2116241501161
Department / Section	Artificial Intelligence and Machine Learning	Date of Viva Demo Duration	To be completed
Application URL Verified	To be completed		To be completed

Table F.1: Viva-Voce Questions and Responses

Q.No.	***Question / Topic Asked***	***Unit***	***Response (E / G / S / NI)***	***Remarks***
1				
2				
3				
4				
5				
6				
7				
8				

E –Excellent G – Good S – Satisfactory NI–NeedsImprovement

Table F.2: Marks Summary

Component	***Max. Marks***	***Marks Awarded***
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	***100***	

Examiners’ Remarks:

_____

_____

INTERNAL EXAMINER ***EXTERNAL EXAMINER***

Name: _____ Name: _____